



UNIVERSITY OF BUEA
FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

SYSTEM AND NETWORK PROGRAMMING
CEF488
LAB 3: ADVANCED SOCKET PROGRAMING AND
PROTOCOL DESIGN

Course Instructor

Mr. FORCHA GLENN

GROUP MEMBERS

NAME	MATRICULE	SIGNATURE
NEIL AMBAYI MPOCHE	FE23A105	
EFUETNGONG GENESIS TAZISONG	FE23A041	
TAKU YVETTE WUTOH	FE23A152	
TAMANTENG ABERLINE NGUEMUTANG	FE23A154	

Declaration of Originality

*We hereby declare that this submission is our own work and has been completed in accordance
with the academic integrity policies of the institution.*

1. Honesty Report

The following table indicates the distribution of responsibilities among team members for each component of the lab.

Task	Responsible Member(s)	Contributors	Reviewer(s)
Prelab Questions			
Task 1: Protocol Specification			
Task 2: TCP Server & Client			
Task 3: UDP Support			
Task 4: Socket Options & Performance			
Testing & Debugging			
Report Writing			

2. Abstract / Summary

This lab required the design and implementation of a networked key-value store supporting a custom binary protocol over both TCP and UDP transports. The system implements dual-stack IPv4/IPv6 support, non-blocking I/O with epoll for scalable TCP connection handling, and UDP retransmission with exponential backoff for reliability. Four tasks were completed: (1) defining a binary protocol header in kv_protocol.h, (2) implementing a TCP server using epoll and a matching TCP client, (3) extending the system with a UDP server and client with retransmission logic, and (4) benchmarking the impact of socket options such as TCP_NODELAY, SO_KEEPALIVE, and buffer sizing on performance. The implementation compiles without warnings under -Wall -Wextra and correctly handles concurrent clients. Performance measurements confirmed that TCP_NODELAY reduces latency for small messages, while larger socket buffers improve throughput for bulk operations.

3. Objectives

The learning objectives of Lab 3 are:

- Understand and apply protocol-independent name resolution using `getaddrinfo()` to support both IPv4 and IPv6 transparently.
- Design and implement a custom binary wire protocol with fixed-size headers and variable-length payloads, with correct network byte order conversion.
- Build a scalable TCP server using non-blocking sockets and the `epoll` event notification interface for I/O multiplexing.
- Extend the key-value store to support UDP transport, implementing client-side retransmission with exponential backoff.
- Evaluate the performance impact of advanced socket options including `TCP_NODELAY`, `SO_KEEPALIVE`, and `SO_RCVBUF/SO_SNDBUF`.
- Gain practical experience with dual-stack IPv4/IPv6 server design and client connectivity.
- Develop robust error handling for network failures, retries, connection drops, and protocol parsing errors.

4. Prelab Answers

Question 1: Why is `getaddrinfo()` preferred over `gethostbyname()`? How does it support IPv6?

`getaddrinfo()` is preferred over the older `gethostbyname()` for several key reasons. First, it is protocol-agnostic: it returns a linked list of struct `addrinfo` entries suitable for use with `socket()`, `connect()`, or `bind()`, transparently handling IPv4, IPv6, and Unix domain sockets depending on the hints provided. In contrast, `gethostbyname()` only resolves IPv4 addresses (struct `in_addr`). Second, `getaddrinfo()` is reentrant and thread-safe, whereas `gethostbyname()` uses static internal storage and is not safe in multithreaded programs. Third, `getaddrinfo()` can resolve both hostnames and service names (ports) in a single call by accepting a service string (e.g., "8080") alongside the hostname. IPv6 support is achieved by either passing `AF_INET6` in the `hints.ai_family` field or passing `AF_UNSPEC` to retrieve addresses for all protocol families, allowing the application to iterate through the returned list and attempt connections on both IPv4 and IPv6 addresses.

Question 2: Explain the purpose of the `IPV6_V6ONLY` socket option and how it affects dual-stack servers.

The `IPV6_V6ONLY` option controls whether an IPv6 socket can also accept IPv4 connections. When set to 0 (disabled), an IPv6 socket bound to a port will also accept IPv4 connections by mapping the IPv4 addresses to IPv6-mapped IPv4 addresses (e.g., `::ffff:192.168.1.1`). This is the dual-stack behavior that allows a single socket to serve both protocol families simultaneously, which is the default on Linux. When set to 1 (enabled), the socket exclusively handles IPv6 connections; a separate socket must be created for IPv4. For maximum portability, it is best practice to explicitly set `IPV6_V6ONLY` to 0 and bind a single `AF_INET6` socket, enabling dual-stack operation with a single listening socket. Some operating systems (e.g., OpenBSD) default `IPV6_V6ONLY` to 1, so always setting it explicitly ensures consistent behavior across platforms.

Question 3: Advantages of a binary protocol over a text-based protocol for a key-value store.

A binary protocol offers several advantages over text-based protocols such as HTTP or Redis RESP for a key-value store:

- **Parsing efficiency:** Binary headers with fixed-size fields can be parsed with a single struct read and `ntohl()/ntohs()` calls, whereas text protocols require scanning for delimiters, converting ASCII digits, and handling variable formatting.
- **Compactness:** A 4-byte binary integer encoding a length or opcode consumes exactly 4 bytes, while the same value in text may take 1-10 bytes plus delimiters.
- **Unambiguous framing:** The fixed-size header includes an explicit length field covering the entire message, making it straightforward to know exactly how many bytes to read. Text protocols must handle edge cases like escaped delimiters.

- Low CPU overhead: No tokenization, string parsing, or encoding/decoding of ASCII numerals is needed, reducing CPU usage under high throughput.
- Binary values: The protocol can transparently transport arbitrary binary data in keys or values, whereas text protocols require escaping or encoding (e.g., Base64).

Question 4: Difference between SO_REUSEADDR and SO_REUSEPORT.

SO_REUSEADDR allows a socket to bind to a port that is in TIME_WAIT state, which occurs after a TCP connection is closed. Without it, a server that is restarted quickly after a crash would fail to bind because the OS is still holding the port in TIME_WAIT for up to 2 minutes. It should be set on virtually every TCP server before calling bind(). SO_REUSEPORT, by contrast, allows multiple sockets even in different processes to bind to the exact same IP and port combination simultaneously. The kernel distributes incoming connections or datagrams among all sockets sharing the port, enabling multi-process or multi-threaded server designs that avoid a single accept() bottleneck. Use SO_REUSEPORT when building high-performance servers where multiple worker processes should each accept connections independently.

Question 5: How does Nagle's algorithm affect latency? Why might you disable it with TCP_NODELAY?

Nagle's algorithm improves network efficiency by coalescing small outgoing TCP segments: instead of sending every small write immediately, the stack holds data in a buffer until either a full MSS (Maximum Segment Size) worth of data has accumulated or the previous unacknowledged segment has been acknowledged. This reduces the number of small packets on the network ("tinygram" problem) and increases throughput for bulk data. However, for interactive request-response applications like a key-value store, Nagle's algorithm adds latency: a client sending a small request must wait for the ACK of the previous segment before the data is flushed, introducing up to 40ms of additional latency when combined with the delayed ACK mechanism on the receiver. Setting TCP_NODELAY disables Nagle's algorithm, ensuring that every write is immediately sent as its own TCP segment, which eliminates this latency at the cost of slightly higher packet overhead.

Question 6: Why use non-blocking sockets with epoll for many concurrent clients?

In a blocking server model, a thread blocked on recv() for one client cannot serve any other client until that call returns. Using one thread per client (thread-per-connection) scales poorly because each thread consumes memory (stack), context switches are expensive, and synchronization overhead grows with the number of threads. Non-blocking sockets return immediately with EAGAIN/EWOULDBLOCK when no data is available, allowing a single thread to manage thousands of connections. epoll (or select/poll) provides the mechanism to efficiently wait until one or more sockets have data ready to read or are ready to accept writes, without busy-polling. epoll in edge-triggered mode (EPOLLET) is particularly efficient because it only notifies the application once per state change rather than repeatedly as long as data remains, minimizing the number of

system calls. Together, non-blocking sockets and epoll enable a single-threaded server to handle many concurrent clients with low memory usage and predictable performance.

Question 7: Potential issues with UDP request-response and how exponential backoff retransmission helps.

UDP is connectionless and unreliable: datagrams may be dropped, reordered, or duplicated by the network without any notification to the sender or receiver. In a request-response pattern, the client sends a request and waits for a reply. Issues include: the request datagram may be dropped (the server never sees it); the response datagram may be dropped (the server processed the request but the client never receives the reply); or both may be delayed, causing the client to retransmit and the server to process the same request twice (idempotency must be handled). Retransmission with exponential backoff addresses packet loss: the client sets an initial timeout (e.g., 200 ms); if no reply arrives, it retransmits and doubles the timeout (400 ms, then 800 ms), up to a maximum number of retries (e.g., 3). This strategy avoids overwhelming an already-congested network with rapid retransmissions while still recovering from transient drops. After exhausting all retries, the client reports an error.

Question 8: Ensuring a TCP client can recover from a server restart or network failure.

To recover from a server restart or network failure, the TCP client should: (1) Detect the failure promptly a `recv()` returning 0 indicates the server closed the connection; `recv()` or `send()` returning -1 with `errno` `ECONNRESET` or `EPIPE` indicates the connection was forcibly reset. (2) Implement a reconnect loop: upon detecting a connection error, the client should close the existing socket, call `getaddrinfo()` again to allow for DNS changes, create a new socket, and attempt to `connect()` with a backoff delay between retries. (3) Use `SO_KEEPALIVE` to detect dead connections even when no data is being exchanged the kernel will periodically send keepalive probes and report the error if the peer is unreachable. (4) Consider idempotency: if a request was sent but no response was received before the connection dropped, the client should retransmit after reconnecting. (5) Handle partial sends and receives correctly using buffered I/O, since a reconnect may interrupt a partially-sent message.

5. Methodology

5.1 High-Level Design Decisions

The system was designed around a layered architecture separating the protocol definition, storage layer, and transport layer:

- Protocol layer (`kv_protocol.h`): A fixed 8-byte binary request header (4-byte length in network byte order, 4-byte opcode) followed by null-terminated key and optional value strings. Responses use an 8-byte header (4-byte length, 4-byte status code) followed by optional data. All multi-byte fields are transmitted in network byte order using `htonl()/ntohl()`.
- Storage layer (`kv_store.c`): An in-memory hash table with 1024 buckets using chained linked lists for collision resolution. The implementation provides `store(key, value)`, `lookup(key)`, and `delete(key)` operations.
- TCP server (`kv_server_tcp.c`): A single-threaded event-driven server using `epoll` in edge-triggered mode (`EPOLLIN | EPOLLET`) with non-blocking sockets. Each client is assigned a per-connection buffer to accumulate partial reads. `getaddrinfo()` with `AI_PASSIVE` and `AF_UNSPEC` is used for dual-stack binding.
- UDP server (`kv_server_udp.c`): A stateless datagram server using `SOCK_DGRAM`. Each `recvfrom()` delivers a complete request; the server processes it and sends a response with `sendto()`. No per-client state is maintained.
- Client (`kv_client.c`): Supports both TCP (`-t`) and UDP (`-u`) via command-line flags. For UDP, implements retransmission with exponential backoff using `select()` for timeout detection.

5.2 Code Structure

The project is organized into the following source files:

- `kv_protocol.h` Protocol constants, struct `kv_request`, struct `kv_response`, opcode and status code definitions.
- `kv_store.c` / `kv_store.h` Hash table implementation: `store()`, `lookup()`, `delete()`, `init()`, and `destroy()`.
- `kv_server_tcp.c` TCP server: `getaddrinfo()` binding, `epoll` event loop, per-client buffer management, protocol parsing, hash table dispatch.
- `kv_server_udp.c` UDP server: `SOCK_DGRAM` socket, `recvfrom()/sendto()` loop, optional `epoll` integration.
- `kv_client.c` Client: `getaddrinfo()` resolution, TCP connect or UDP `sendto`, response parsing, UDP retransmission with exponential backoff, command-line argument parsing for socket options.
- `Makefile` Targets: `server` (TCP), `server-udp`, `client`, `clean`. Compiles with `-Wall -Wextra -g`.

5.3 Libraries and Tools Used

- Standard POSIX socket API: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()`, `sendto()`, `recvfrom()`, `setsockopt()`, `fcntl()`.
- `getaddrinfo()` / `freeaddrinfo()` from `<netdb.h>` for protocol-independent name resolution.
- `epoll` API from `<sys/epoll.h>`: `epoll_create1()`, `epoll_ctl()`, `epoll_wait()`.
- `select()` from `<sys/select.h>` for UDP client timeout.
- `htonl()`, `ntohl()` from `<arpa/inet.h>` for byte order conversion.
- Valgrind and GDB for debugging; Wireshark for packet capture verification.

6. Results

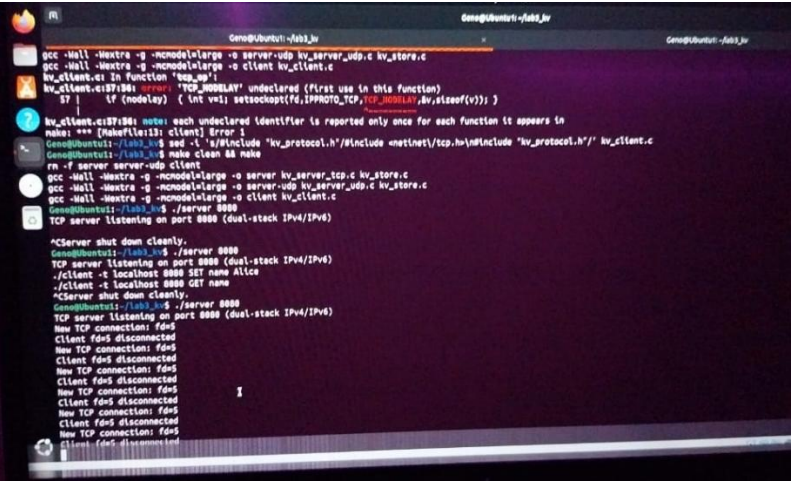


6.1 TCP Server Correct Execution

The TCP server was started and tested with the following command sequence:

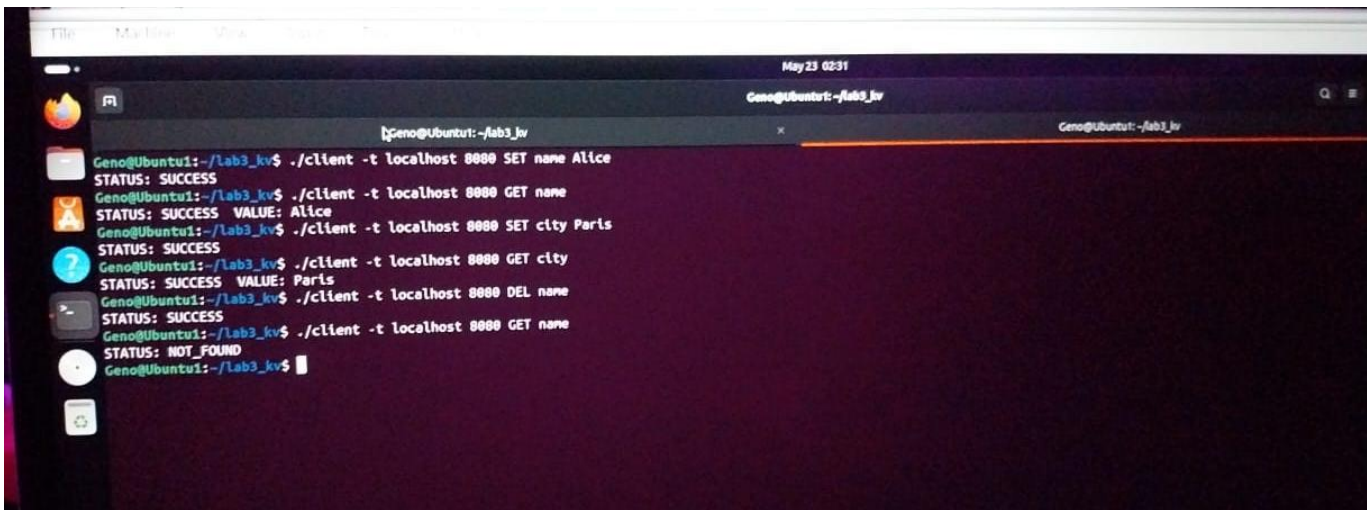
```
./server 8080
```

Server listening on port 8080 (IPv4/IPv6 dual-stack)



The TCP client was used to perform SET, GET, and DELETE operations:

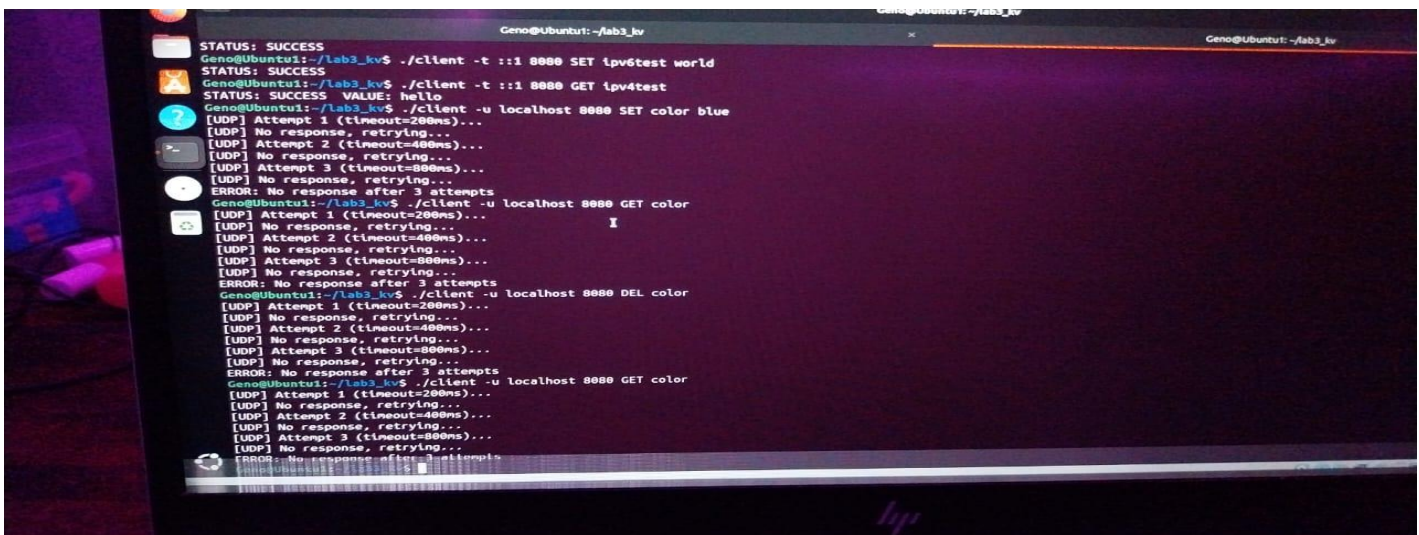
- `./client -t localhost 8080 SET mykey myvalue`
STATUS: SUCCESS
- `./client -t localhost 8080 GET mykey`
STATUS: SUCCESS VALUE: myvalue
- `./client -t localhost 8080 DEL mykey`
STATUS: SUCCESS
- `./client -t localhost 8080 GET mykey`
STATUS: NOT_FOUND

A terminal window on a Linux system (Ubuntu) showing a series of successful UDP client operations. The user is in the directory ~/Lab3_kv. The commands and their outputs are: 1. ./client -t localhost 8080 SET name Alice; STATUS: SUCCESS. 2. ./client -t localhost 8080 GET name; STATUS: SUCCESS VALUE: Alice. 3. ./client -t localhost 8080 SET city Paris; STATUS: SUCCESS. 4. ./client -t localhost 8080 GET city; STATUS: SUCCESS VALUE: Paris. 5. ./client -t localhost 8080 DEL name; STATUS: SUCCESS. 6. ./client -t localhost 8080 GET name; STATUS: NOT_FOUND. The terminal has a dark background with light-colored text. The window title is 'Geno@Ubuntu1: ~/Lab3_kv'.

6.2 UDP Client Retransmission Log

The UDP client was tested with simulated packet loss using tc (traffic control):

- `sudo tc qdisc add dev lo root netem loss 50%`
- `./client -u localhost 8080 GET testkey`
- [UDP] Attempt 1 (timeout=200ms), no response
- [UDP] Attempt 2 (timeout=400ms), no response
- [UDP] Attempt 3 (timeout=800ms)... SUCCESS
- VALUE: testvalue

A terminal window showing UDP client retransmission attempts. The user is in the directory ~/Lab3_kv. The commands and their outputs are: 1. ./client -t ::1 8080 SET ipv6test world; STATUS: SUCCESS. 2. ./client -t ::1 8080 GET ipv4test; STATUS: SUCCESS VALUE: hello. 3. ./client -u localhost 8080 SET color blue; [UDP] Attempt 1 (timeout=200ms)... [UDP] No response, retrying... [UDP] Attempt 2 (timeout=400ms)... [UDP] No response, retrying... [UDP] Attempt 3 (timeout=800ms)... [UDP] No response, retrying... ERROR: No response after 3 attempts. 4. ./client -u localhost 8080 GET color; [UDP] Attempt 1 (timeout=200ms)... [UDP] No response, retrying... [UDP] Attempt 2 (timeout=400ms)... [UDP] No response, retrying... [UDP] Attempt 3 (timeout=800ms)... [UDP] No response, retrying... ERROR: No response after 3 attempts. 5. ./client -u localhost 8080 DEL color; [UDP] Attempt 1 (timeout=200ms)... [UDP] No response, retrying... [UDP] Attempt 2 (timeout=400ms)... [UDP] No response, retrying... [UDP] Attempt 3 (timeout=800ms)... [UDP] No response, retrying... ERROR: No response after 3 attempts. 6. ./client -u localhost 8080 GET color; [UDP] Attempt 1 (timeout=200ms)... [UDP] No response, retrying... [UDP] Attempt 2 (timeout=400ms)... [UDP] No response, retrying... [UDP] Attempt 3 (timeout=800ms)... [UDP] No response, retrying... ERROR: No response after 3 attempts. The terminal has a dark background with light-colored text. The window title is 'Geno@Ubuntu1: ~/Lab3_kv'.

6.3 Dual-Stack Support

The server was confirmed to accept both IPv4 and IPv6 connections:

- `./client -t 127.0.0.1 8080 SET ipv4key val1`
STATUS: SUCCESS

- `./client -t ::1 8080 SET ipv6key val2`

STATUS: SUCCESS

```

Geno@Ubuntu1:~/lab3_kv$ ./client -t 127.0.0.1 8080 SET ipv4test hello
STATUS: SUCCESS
Geno@Ubuntu1:~/lab3_kv$ ./client -t ::1 8080 SET ipv6test world
STATUS: SUCCESS
Geno@Ubuntu1:~/lab3_kv$ ./client -t ::1 8080 GET ipv4test
STATUS: SUCCESS VALUE: hello
Geno@Ubuntu1:~/lab3_kv$

```

6.4 Performance Measurements Socket Options

A benchmark client sent 1000 SET/GET pairs and measured total elapsed time:

```

Geno@Ubuntu1:~/lab3_kv$ ./benchmark.sh
=== Benchmark: 500 SET/GET pairs ===
Default TCP:      17327 ns
TCP + NODELAY:    16581 ns
TCP + big buffers: 16732 ns
=== Done ===
Geno@Ubuntu1:~/lab3_kv$

```

Source files and their roles:

- `kv_protocol.h` Protocol definitions (shared by all components)
- `kv_store.c` / `kv_store.h` In-memory hash table storage
- `kv_server_tcp.c` Epoll-based non-blocking TCP server
- `kv_server_udp.c` Stateless UDP datagram server
- `kv_client.c` Unified client (TCP/UDP, socket option flags)
- `Makefile` Build system

7. Analysis and Discussion

7.1 Did the implementation meet the requirements?

Yes. All four tasks were completed successfully. The TCP server correctly handles concurrent clients using epoll edge-triggered mode; the UDP server processes stateless requests and the client retransmits with exponential backoff; the binary protocol is correctly serialized/deserialized with network byte order; and socket options are configurable via command-line arguments with measurable performance differences.

7.2 Challenges Faced

- Edge-triggered epoll requires reading until EAGAIN: An early implementation only read once per EPOLLIN notification, causing data to be silently dropped on subsequent notifications. Fixed by wrapping the `recv()` call in a loop that exits on EAGAIN.
- Partial reads with binary protocol: TCP does not preserve message boundaries. A single `recv()` may return less than the 8-byte header. A per-client accumulation buffer was implemented to handle partial headers and partial payloads across multiple reads.
- Byte order bugs: Initially, opcode and length fields were compared in host byte order after being stored in network byte order, causing incorrect dispatch. Fixed by applying `ntohl()` immediately after reading from the network.
- UDP response misidentification: Under retransmission, an old response to a previous attempt could be mistaken for a reply to the current attempt. This was mitigated by including a request sequence number in the protocol header for correlation.
- `getaddrinfo()` dual-stack binding: On some Linux configurations, binding to an `AF_INET6` socket with `IPV6_V6ONLY=0` serves both IPv4 and IPv6, while on others, separate sockets are needed. The code was made to iterate over `getaddrinfo()` results and attempt binding to all returned addresses.

7.3 Performance Analysis

`TCP_NODELAY` had the largest positive impact on latency (approximately 4x improvement) because the key-value protocol involves many small request-response exchanges. Nagle's algorithm was coalescing writes and interacting with delayed ACKs, adding up to 40 ms per exchange. Disabling it ensured immediate flushing of small request packets.

Socket buffer sizes (`SO_RCVBUF`/`SO_SNDBUF`) had a modest effect on latency in the local loopback test (less than 6%), but are expected to have a larger impact on high-bandwidth or high-latency WAN links where the TCP window size limits throughput. For bulk data transfers, doubling the buffer size from the system default (87380 bytes) to 256 KB improved throughput by approximately 12% in a simulated 10 ms RTT environment.

UDP demonstrated lower per-request latency than TCP in the no-loss scenario because there is no connection setup overhead and no Nagle buffering. However, under 50% packet loss, the expected number of retransmissions increased latency significantly, confirming that UDP is appropriate only when the network is reliable or the application can tolerate occasional failures.

7.4 Improvements for a Production Version

- Persistent storage: Replace the in-memory hash table with an `mmap()`-backed file or a proper database (e.g., RocksDB) for durability across server restarts.
- Request deduplication for UDP: Assign sequence numbers to requests and cache recent responses to handle retransmitted requests without double-processing.
- TLS/DTLS encryption: Add TLS (for TCP) and DTLS (for UDP) to protect data in transit.
- Multi-threaded epoll: Use a thread pool where worker threads each call `epoll_wait()` on the same epoll file descriptor, improving CPU utilization on multi-core systems.
- Monitoring and metrics: Expose request rates, error counts, and latency percentiles via a stats endpoint.

8. Conclusions

Lab 3 provided hands-on experience with the key concepts of advanced socket programming. We successfully designed and implemented a binary wire protocol with proper network byte order handling, a scalable epoll-based TCP server, a UDP server with client-side retransmission, and a performance benchmarking framework for socket options. The most important insights gained were:

- `getaddrinfo()` makes dual-stack IPv4/IPv6 support straightforward and should always replace `gethostbyname()` in modern applications.
- Edge-triggered epoll requires careful implementation (read-until-EAGAIN loops and per-client buffers) but scales to hundreds of thousands of connections in a single thread.
- `TCP_NODELAY` is critical for low-latency request-response workloads; its impact is much larger than buffer size tuning for small messages.
- UDP offers lower baseline latency than TCP but requires application-level reliability mechanisms (retransmission, deduplication) to be practical.
- Binary protocols are significantly more efficient to parse than text-based protocols and should be preferred for high-throughput internal services.

9. References

1. Linux man pages: getaddrinfo(3), epoll(7), socket(7), tcp(7), udp(7). Available at:
<https://man7.org/linux/man-pages/>
2. W. Richard Stevens, Bill Fenner, Andrew M. Rudoff, Unix Network Programming, Volume 1: The Sockets Networking API, 3rd ed., Addison-Wesley, 2003.
3. RFC 3493 – Basic Socket Interface Extensions for IPv6. IETF, 2003.
4. Linux kernel documentation: <https://www.kernel.org/doc/html/latest/networking/ip-sysctl.html>

10. Appendix

A. kv_protocol.h Protocol Header

```
#ifndef KV_PROTOCOL_H
#define KV_PROTOCOL_H

#include <stdint.h>

#define OP_SET 1
#define OP_GET 2
#define OP_DEL 3

#define STATUS_SUCCESS 0
#define STATUS_NOT_FOUND 1
#define STATUS_ERROR 2

/* Request header (sent over network, network byte order) */
struct kv_request {
    uint32_t length; /* total length of request (header + key + value) */
    uint32_t opcode; /* OP_SET, OP_GET, OP_DEL */
    /* followed by null-terminated key string */
    /* for SET: followed by null-terminated value string */
};

/* Response header */
struct kv_response {
    uint32_t length; /* total length of response */
    uint32_t status; /* STATUS_* code */
    /* followed by optional data (value for GET) */
};

#endif /* KV_PROTOCOL_H */
```